Module 09 - Object-Oriented Programming

Table of Contents

- [ ] 1. What is OOP? — The Blueprint Analogy
- [ ] 2. Your First Class
- [ ] 3. The Constructor — `__init__`
- [ ] 4. Instance Attributes
- [ ] 5. Methods — What Can the Object Do?
- [ ] 6. The `__str__` Method — Human-Readable Output
- [ ] 7. Putting It Together — A Security Tool Object
- [ ] 8. Summary and Next Steps

---

1. What is OOP? — The Blueprint Analogy

Imagine you are a security engineer and you need to track **vulnerabilities**. Every vulnerability has the same kind of information: a CVE ID, a severity score, an affected system, and whether it has been patched.

You could create 4 separate variables for each vulnerability... but you have 200 vulnerabilities. That is 800 variables. Chaos.

Object-Oriented Programming (OOP) solves this by letting you define a **blueprint** — called a **class** — that describes what a vulnerability looks like. Then you create as many **objects** (instances) from that blueprint as you need, each with its own data.

```
CLASS (Blueprint)         OBJECT (Real instance)
------------------        --------------------------------
Vulnerability  ------>    CVE-2023-44487  (HTTP/2 Rapid Reset)
               ------>    CVE-2021-44228  (Log4Shell)
               ------>    CVE-2023-23397  (Outlook RCE)
```

The blueprint defines the **structure**. The objects hold the **real data**.

Key vocabulary

Term	Plain English	Example
Class	The blueprint / template	`Vulnerability`

Term	Plain English	Example
Object / Instance	One real thing built from the blueprint	`CVE-2021-44228`
Attribute	A piece of data that belongs to the object	`cvss_score = 10.0`
Method	A function that belongs to the object	`is_critical()`

---

2. Your First Class

The syntax to define a class is:

```
class ClassName:
    """Docstring: describe what this class represents"""
    pass  # placeholder — we'll add content soon
```

Rules:

- Class names use **PascalCase** (each word capitalised, no underscores): `Vulnerability`, `FirewallRule`, `LogEntry`
- Functions use `snake_case` — classes are the exception
- The `class` keyword is followed by the name and a colon
- Everything inside is indented (same rule as `if` and `def`)

In [ ]:
```
# Our first class — just the shell
# We'll fill it in over the next sections

class Vulnerability:
    """Represents a security vulnerability tracked by the SOC team."""
    pass  # empty for now


# Creating an object (instance) from the class:
vuln_1 = Vulnerability()  # Vulnerability() calls the class like a function
vuln_2 = Vulnerability()  # a completely separate object

print(type(vuln_1))   # confirms it is a Vulnerability object
print(vuln_1 is vuln_2)  # False — they are two different objects in memory
```

🎯 Practice — Your First Class

Exercise 1 — Write

Create an empty class called `FirewallRule`. Then create two instances of it: `rule_1` and `rule_2`. Print the type of `rule_1` to confirm it was created correctly.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

3. The Constructor — `__init__`

Right now our class is an empty shell. We need a way to give each object its own data when it is created. That is the job of the **constructor**: a special method called `__init__`.

`__init__` runs **automatically** every time you create a new object.

```
class ClassName:
    def __init__(self, param1, param2):
        self.param1 = param1   # store as attribute on this object
        self.param2 = param2
```

What is `self`?

Think of `self` as **the object referring to itself**. When you write `self.cvss_score`, you are saying: "attach the attribute `cvss_score` to *this specific object*".

- `self` is **always the first parameter** in any method — but you never pass it yourself
- Python passes it automatically when you call the method

```
Vulnerability("CVE-2021-44228", 10.0)
      |                |         |
    class            cve_id   cvss_score
                     (self is handled automatically)
```

In [ ]:

```
# Topic: The __init__ constructor — giving each object its own data

class Vulnerability:
    """Represents a security vulnerability tracked by the SOC team."""

    def __init__(self, cve_id, cvss_score, affected_system, is_patched):
        """Called automatically when a Vulnerability object is created."""
        self.cve_id = cve_id                  # e.g. 'CVE-2021-44228'
        self.cvss_score = cvss_score           # float 0.0-10.0
        self.affected_system = affected_system  # e.g. 'Apache Log4j'
        self.is_patched = is_patched           # True/False


# Creating two vulnerability objects — each holds its own separate data
log4shell = Vulnerability("CVE-2021-44228", 10.0, "Apache Log4j", False)
outlook_rce = Vulnerability("CVE-2023-23397", 9.8, "Microsoft Outlook", True)

# Accessing attributes with dot notation
print(log4shell.cve_id)          # CVE-2021-44228
print(log4shell.cvss_score)      # 10.0
print(outlook_rce.is_patched)    # True
```

🎯 Practice — The Constructor

Exercise 2 — Write

Create a class `NetworkPort` with the following attributes set in `__init__`:

- `port_number` (int) — e.g. 443
- `protocol` (str) — e.g. `'TCP'`
- `service_name` (str) — e.g. `'HTTPS'`
- `is_open` (bool) — whether the port is currently open

Then create two instances: port 22 (SSH, TCP, open) and port 3306 (MySQL, TCP, closed). Print the `service_name` and `is_open` of each.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

Exercise 3 — Predict

Before running the code below, predict what each `print` will output. Think carefully about which object each attribute belongs to.

```
class LogEntry:
    def __init__(self, source_ip, event_type, severity):
        self.source_ip = source_ip
        self.event_type = event_type
        self.severity = severity

entry_a = LogEntry("10.0.0.14", "FAILED_LOGIN", "HIGH")
entry_b = LogEntry("192.168.1.5", "PORT_SCAN", "MEDIUM")

print(entry_a.source_ip)
print(entry_b.severity)
print(entry_a.event_type == entry_b.event_type)
```

Write your predictions here before running:

1. `entry_a.source_ip` → ?
2. `entry_b.severity` → ?
3. `entry_a.event_type == entry_b.event_type` → ?

In [ ]:
```
# Predict first, then run to check!

class LogEntry:
    def __init__(self, source_ip, event_type, severity):
        self.source_ip = source_ip
        self.event_type = event_type
        self.severity = severity
```

```
entry_a = LogEntry("10.0.0.14", "FAILED_LOGIN", "HIGH")
entry_b = LogEntry("192.168.1.5", "PORT_SCAN", "MEDIUM")

print(entry_a.source_ip)                          # ?
print(entry_b.severity)                           # ?
print(entry_a.event_type == entry_b.event_type)   # ?
```

---

4. Instance Attributes

Every attribute you set with `self.something = value` is an **instance attribute** — it belongs to that specific object and is completely independent from other objects.

You can also **modify** attributes after creation using dot notation:

`object.attribute = new_value`
This is useful in security workflows — for example, when you patch a vulnerability, you update `is_patched` from `False` to `True`.

In [ ]:
```
# Topic: Reading and modifying instance attributes

class Vulnerability:
    def __init__(self, cve_id, cvss_score, affected_system, is_patched):
        self.cve_id = cve_id
        self.cvss_score = cvss_score
        self.affected_system = affected_system
        self.is_patched = is_patched


log4shell = Vulnerability("CVE-2021-44228", 10.0, "Apache Log4j", False)

print("Before patch:", log4shell.is_patched)   # False

# The patch has been applied — update the attribute
log4shell.is_patched = True

print("After patch: ", log4shell.is_patched)   # True

# You can also update numeric attributes
# (e.g. CVSS score revised by NVD after initial publication)
log4shell.cvss_score = 10.0   # confirmed critical
print("CVSS score:  ", log4shell.cvss_score)
```

🎯 Practice — Modifying Attributes

Exercise 4 — Write

A `UserAccount` class tracks system users. Create it with attributes:

- `username` (str)

- `failed_attempts` (int) — starts at 0
- `is_locked` (bool) — starts as `False`

Then:

1. Create an account for `'jdoe'`
2. Simulate 3 failed logins by adding 1 to `failed_attempts` three times
3. Lock the account by setting `is_locked` to `True`
4. Print the final state of all three attributes

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

5. Methods — What Can the Object Do?

Attributes describe **what an object IS**. Methods describe **what an object can DO**.

A method is just a function defined inside a class. The only rule: it must always have `self` as its first parameter — this gives it access to the object's attributes.

```
class ClassName:
    def __init__(self, ...):
        self.attribute = value

    def my_method(self):      # <-- self gives access to attributes
        # do something with self.attribute
        return result
```

Methods are called using **dot notation** on the object:

```
result = my_object.my_method()
```

In [ ]:
```
# Topic: Adding methods to a class

class Vulnerability:
    """Represents a security vulnerability tracked by the SOC team."""

    def __init__(self, cve_id, cvss_score, affected_system, is_patched):
        self.cve_id = cve_id
        self.cvss_score = cvss_score
        self.affected_system = affected_system
        self.is_patched = is_patched

    def get_severity(self):
        """Returns severity label based on CVSS score (NIST scale)."""
        if self.cvss_score >= 9.0:
            return "CRITICAL"
        elif self.cvss_score >= 7.0:
```

```
            return "HIGH"
        elif self.cvss_score >= 4.0:
            return "MEDIUM"
        else:
            return "LOW"

    def needs_urgent_action(self):
        """Returns True if the vuln is CRITICAL or HIGH and not yet patched."""
        is_severe = self.cvss_score >= 7.0   # HIGH or CRITICAL
        return is_severe and not self.is_patched


# Testing our methods
log4shell = Vulnerability("CVE-2021-44228", 10.0, "Apache Log4j", False)
low_risk = Vulnerability("CVE-2023-00001", 3.2, "Internal tool", False)

print(log4shell.get_severity())          # CRITICAL
print(log4shell.needs_urgent_action())   # True  (critical + not patched)
print(low_risk.needs_urgent_action())    # False (low severity)
```

🎯 Practice — Methods

Exercise 5 — Write

Add a method `should_lock(self)` to the `UserAccount` class below. It should return `True` if `failed_attempts` is 3 or more **and** the account is not already locked. Print the result for both accounts.

In [ ]:
```
class UserAccount:
    def __init__(self, username, failed_attempts, is_locked):
        self.username = username
        self.failed_attempts = failed_attempts
        self.is_locked = is_locked

    # Add your method here


suspicious = UserAccount("attacker99", 5, False)
normal = UserAccount("alice", 1, False)

# Your print statements here
```

▶ 💡 **SOLUTION**

Exercise 6 — Fix

The code below has a bug. The `is_dangerous_port` method is supposed to return `True` for ports that are commonly targeted (21, 23, 3389, 3306). Find and fix the bug.

```
class NetworkPort:
    def __init__(self, port_number, service_name, is_open):
        self.port_number = port_number
```

```
        self.service_name = service_name
        self.is_open = is_open

    def is_dangerous_port(self):
        dangerous_ports = [21, 23, 3389, 3306]
        return port_number in dangerous_ports  # <-- bug here

ftp = NetworkPort(21, 'FTP', True)
print(ftp.is_dangerous_port())
```

In [ ]:

```
# Fix the buggy code below:

class NetworkPort:
    def __init__(self, port_number, service_name, is_open):
        self.port_number = port_number
        self.service_name = service_name
        self.is_open = is_open

    def is_dangerous_port(self):
        dangerous_ports = [21, 23, 3389, 3306]
        return port_number in dangerous_ports  # <-- bug here

ftp = NetworkPort(21, "FTP", True)
print(ftp.is_dangerous_port())
```

▶ 💡 **SOLUTION**

---

6. The `__str__` Method — Human-Readable Output

Right now, if you print a `Vulnerability` object, Python shows something like: `<__main__.Vulnerability object at 0x7f3a2b...>` — completely unreadable.

The `__str__` method lets you define what Python should print when you do `print(my_object)`. It must always return a **string**.

```
def __str__(self):
    return f"some string with {self.attributes}"
```

This is used for example for log formatters, alert summaries, vulnerability reports all rely on clean string representations of objects.

In [ ]:

```
# Topic: __str__ for readable security tool output

class Vulnerability:
    def __init__(self, cve_id, cvss_score, affected_system, is_patched):
        self.cve_id = cve_id
        self.cvss_score = cvss_score
        self.affected_system = affected_system
```

```
        self.is_patched = is_patched

    def get_severity(self):
        if self.cvss_score >= 9.0:
            return "CRITICAL"
        elif self.cvss_score >= 7.0:
            return "HIGH"
        elif self.cvss_score >= 4.0:
            return "MEDIUM"
        else:
            return "LOW"

    def __str__(self):
        """Returns a one-line summary suitable for a SOC report."""
        patch_status = "PATCHED" if self.is_patched else "UNPATCHED"
        return (
            f"[{self.get_severity()}] {self.cve_id} | "
            f"CVSS: {self.cvss_score} | "
            f"System: {self.affected_system} | "
            f"Status: {patch_status}"
        )


log4shell = Vulnerability("CVE-2021-44228", 10.0, "Apache Log4j", False)
print(log4shell)
# [CRITICAL] CVE-2021-44228 | CVSS: 10.0 | System: Apache Log4j | Status: UNPATCHED
```

🎯 Practice — `__str__`

Exercise 7 — Write

Add a `__str__` method to the `FirewallRule` class below. It should return a string in this format:

```
ALLOW TCP 443 (HTTPS) from 0.0.0.0/0
```

or

```
DENY TCP 23 (Telnet) from 0.0.0.0/0
```

Use the attributes: `action`, `protocol`, `port`, `service_name`, `source`.

In [ ]:
```
class FirewallRule:
    def __init__(self, action, protocol, port, service_name, source):
        self.action = action           # 'ALLOW' or 'DENY'
        self.protocol = protocol       # 'TCP' or 'UDP'
        self.port = port               # port number
        self.service_name = service_name
        self.source = source           # IP range or '0.0.0.0/0' for any

    # Add __str__ here
```

```
https_rule = FirewallRule("ALLOW", "TCP", 443, "HTTPS", "0.0.0.0/0")
telnet_rule = FirewallRule("DENY", "TCP", 23, "Telnet", "0.0.0.0/0")

print(https_rule)
print(telnet_rule)
```

▶ 💡 **SOLUTION**

---

7. Putting It Together — A Security Tool Object

Let's build a more complete object that combines everything: constructor, attributes, multiple methods, and `__str__`.

This is the kind of class you would write in a real SOC tool.

In [ ]:
```
# Topic: Full class combining all ELI5 concepts
# Real-world scenario: a basic alert object for a SIEM-like tool

class SecurityAlert:
    """Represents a security alert raised by detection rules."""

    def __init__(self, alert_id, source_ip, rule_name, severity, is_acknowledged):
        self.alert_id = alert_id              # unique identifier
        self.source_ip = source_ip            # attacker or suspicious IP
        self.rule_name = rule_name            # detection rule that fired
        self.severity = severity              # 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL'
        self.is_acknowledged = is_acknowledged  # analyst has seen it

    def acknowledge(self):
        """Mark this alert as seen by an analyst."""
        self.is_acknowledged = True

    def is_high_priority(self):
        """Returns True if this alert needs immediate analyst attention."""
        urgent_levels = ["HIGH", "CRITICAL"]     # LOW and MEDIUM can wait
        return self.severity in urgent_levels and not self.is_acknowledged

    def __str__(self):
        status = "ACK" if self.is_acknowledged else "NEW"
        return (
            f"[{status}][{self.severity}] Alert #{self.alert_id} | "
            f"Rule: {self.rule_name} | Source: {self.source_ip}"
        )


# Simulate a new brute-force alert arriving
alert = SecurityAlert(
    alert_id="ALT-0042",
    source_ip="185.220.101.47",
    rule_name="SSH_BRUTE_FORCE",
```

```
    severity="HIGH",
    is_acknowledged=False
)

print(alert)
print("High priority:", alert.is_high_priority())   # True

alert.acknowledge()
print(alert)
print("High priority:", alert.is_high_priority())   # False (acknowledged)
```

---

8. Summary and Next Steps

Key Rules

Concept	Rule
`class` keyword	Defines the blueprint — uses PascalCase name
`__init__`	Runs automatically on creation — always first method
`self`	Always the first parameter — gives access to the object's own data
`self.attribute`	Stores data on the object — every instance has its own copy
Methods	Functions inside a class — always take `self` as first param
`__str__`	Controls what `print(object)` shows — must return a string

What you can now do

- Design a class with attributes and methods
- Instantiate multiple objects from the same class
- Add logic in methods that uses the object's own data
- Produce clean string output with `__str__`

What comes next

01_Advanced_OOP.ipynb — Encapsulation (protecting data), Inheritance (reusing classes), Static and Class methods